CyberSteer: Hand-Controlled Retro Highway Racer

> **Project Overview & Complete Technical Specification**
>
> *Generated for deep analysis and LLM study (ChatGPT / Claude / Gemini)*
>
> HTML5 Canvas 2D | MediaPipe Hands AI | Computer Vision | Pure Vanilla JS | 16-Bit Pixel Aesthetic

1. Executive Summary

CyberSteer is a browser-based, AI-driven highway racing and obstacle dodging game. It combines real-time client-side computer vision with a high-performance 60 FPS HTML5 2D Canvas game loop.

Players can steer their cybernetic racing car either using their physical hands tracked live via webcam (using Google MediaPipe Hands ML models) or standard keyboard hotkeys (A/D or Arrow keys).

2. Architecture & Technology Stack

Layer	Technology	Purpose
Core Engine	JavaScript (ES6+)	Game physics, state machine, procedural traffic spawner, particle explosion system.
Computer Vision	Google MediaPipe Hands ML + CameraUtils CDN	Real-time single-hand 21-landmark detection, palm centroid tracking, gesture zone classification with hysteresis.
Renderer	HTML5 2D Canvas API	Hardware-accelerated procedural highway rendering, multi-vehicle obstacle rendering, dynamic lighting & headlight projection.
Interface & Styling	HTML5 & Vanilla CSS3	Translucent dark glassmorphism modals, retro 16-bit arcade pixel typography (Press Start 2P), gold/cyan cyber palette.
Storage	Browser LocalStorage	Persistent high-score tracking across sessions.

3. Key Modules & Technical Implementation

3.1. Computer Vision & Gesture Steering (`handControl.js`)

- **MediaPipe Hands Integration:** Initializes an in-browser ML model targeting 21 hand landmarks running asynchronously inside `requestAnimationFrame`.
- **Palm Center Calculation:** Extracts Landmark 9 (Middle Finger MCP), normalized to [0, 1] horizontal coordinates.
- **Mirror Space Transformation:** Converts camera coordinates `rawX = 1 - palmCenter.x` so moving hand right moves the car right from the user's natural perspective.
- **Hysteresis Zone Thresholding:** Divides screen into LEFT (< 0.38), CENTER (0.38 - 0.62), and RIGHT (> 0.62). Implements a ±0.03 hysteresis buffer to eliminate jitter near boundaries.
- **Dynamic Neon Overlay:** Visualizes the hand skeleton with glowing cyan lines, joint nodes, and palm reticle directly over the live camera feed.

3.2. Game Engine & Speed Progression (`game.js`)

- **State Machine:** Controls `MENU`, `PLAYING`, `PAUSED`, and `GAMEOVER` states with smooth modal transitions.
- **Steering Interpolation (Lerp):** Smooths lane changes using linear interpolation `player.x += (targetX - player.x) * 0.18`.
- **Speed Progression:** Starts at 50 mph and progresses through 75 mph, 90 mph, capping strictly at 100 mph. Continuous lerp makes speed transitions feel realistic.
- **Four-Wheel Obstacle Variety:** Procedurally spawns 4 distinct vehicle classes (Cars, SUVs, Pickups, and Semi-Trucks) with custom pixel silhouettes, wheels, and taillights.
- **Particle Engine:** Spawns 30 velocity-driven expanding debris particles with alpha decay on collisions.
- **AABB Collision Detection:** Bounding box collision with customizable padding for forgiving yet precise collision boundaries.

3.3. Responsive Arcade UI & Layout (`index.html` & `style.css`)

- **Dual-Panel Split Screen:** Aligns the left game window and right live camera panel at 470px × 630px side-by-side.
- **Context-Aware Right Panel:** Displays game instructions & controls during Menu mode, and dynamically expands to a full-screen live webcam feed during gameplay.
- **Pixel Font & Hierarchy:** Imports Google Font `Press Start 2P` with gold headings (`#ffd700`), white body text (`#ffffff`), and cyan neon button accents (`#00f2fe`).

4. Complete Source Code Repository

4.1. `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CyberSteer: Hand-Controlled Racing</title>
    <!-- Load Press Start 2P Pixel Font -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Press+Start+2P&display=swap"
rel="stylesheet">
    <!-- Load MediaPipe Hands & Camera CDN dependencies -->
    <script src="https://cdn.jsdelivr.net/npm/@mediapipe/camera_utils/camera_utils.js"
crossorigin="anonymous"></script>
    <script src="https://cdn.jsdelivr.net/npm/@mediapipe/hands/hands.js" crossorigin="anonymous">
</script>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div id="app-container">
        <!-- Game Window (Canvas and Screen Overlays) -->
        <div id="game-window-wrapper">
            <canvas id="gameCanvas" width="470" height="630"></canvas>

            <!-- Game HUD (Floating elements) -->
            <div id="game-hud">
                <div class="hud-panel">
                    <span class="hud-label">Score</span>
                    <span class="hud-val" id="hud-score">0</span>
                </div>
                <div class="hud-panel">
                    <span class="hud-label">Speed</span>
                    <span class="hud-val" id="hud-speed">50 mph</span>
                </div>
            </div>

            <!-- Start Menu Overlay -->
            <div id="start-screen" class="menu-overlay active">
                <h1 class="gold-heading">CyberSteer</h1>
                <p class="subtitle">Hand-Controlled Highway Dodger</p>
                <p style="color: #ffd700; font-size: 0.65rem; line-height: 1.5; margin-bottom:
24px;">Please turn on your camera to use hand control!</p>
                <div class="btn-container">
                    <button id="btn-play-hand" class="cyber-btn primary">Play Hand Mode</button>
                    <button id="btn-play-keyboard" class="cyber-btn secondary">Play Keyboard</button>
                </div>
            </div>

            <!-- Pause Modal Overlay -->
            <div id="paused-screen" class="menu-overlay">
                <h2 class="gold-heading">Game Paused</h2>
                <p class="subtitle">Steering Locked</p>
                <div class="btn-container">
                    <button id="btn-resume" class="cyber-btn primary">Resume</button>
                    <button id="btn-pause-quit" class="cyber-btn secondary btn-quit-action">Quit to
Menu</button>
```

```
                </div>
            </div>

            <!-- Game Over Screen Overlay -->
            <div id="gameover-screen" class="menu-overlay">
                <h2 class="gold-heading" style="text-shadow: 0 0 10px rgba(255, 215, 0, 0.4); margin-
bottom: 8px;">Wrecked</h2>
                <p class="subtitle">Collision Detected</p>
                <div style="margin-bottom: 25px;">
                    <span class="hud-label gold-heading" style="font-size: 0.65rem; display: block;
margin-bottom: 6px;">Final Score:</span>
                    <span id="over-score" style="font-size: 1.6rem; font-weight: 800; display: block;
color: #ffffff; margin-bottom: 12px;">0</span>
                    <span class="hud-label gold-heading" style="font-size: 0.65rem; display: block;
margin-bottom: 6px;">High Score:</span>
                    <span id="over-highscore" style="font-size: 1rem; display: block; color:
#ffffff;">0</span>
                </div>
                <div class="btn-container">
                    <button id="btn-restart" class="cyber-btn primary">Reboot Run</button>
                    <button id="btn-over-quit" class="cyber-btn secondary btn-quit-action">Quit to
Menu</button>
                </div>
            </div>
        </div>

        <!-- Sidebar Panel for How to Play instructions & Live Webcam Feed -->
        <div id="control-sidebar">
            <div class="panel-header">
                <span id="sidebar-title" class="panel-title gold-heading">How to Play</span>
                <span id="control-mode-badge" class="badge keyboard">Keyboard</span>
            </div>

            <!-- Webcam Frame Viewport (Fills panel during gameplay) -->
            <div id="webcam-preview-container">
                <video id="webcam-video" autoplay playsinline muted></video>
                <canvas id="webcam-overlay" width="640" height="480"></canvas>
            </div>

            <!-- Instructions Section (Visible before game starts) -->
            <div id="instructions-section">
                <!-- How to Play instructions list -->
                <div class="how-to-play-container">
                    <ul class="how-to-play-list">
                        <li>Control car with hand or keyboard.</li>
                        <li>Move hand <strong>LEFT</strong> to steer left.</li>
                        <li>Move hand <strong>RIGHT</strong> to steer right.</li>
                        <li>Keep hand in <strong>CENTER</strong> to go straight.</li>
                        <li>Avoid cars and trucks.</li>
                        <li>Speed starts at 50 mph & increases to 100 mph.</li>
                        <li>Survive to maximize your score!</li>
                    </ul>
                </div>

                <!-- Keybind Controls Reference -->
                <div class="keyboard-controls-guide">
                    <div class="guide-title gold-heading">Controls</div>
                    <div class="ctrl-row">
                        <span>A / Left</span>
                        <span class="key-cap">Steer L</span>
                    </div>
                    <div class="ctrl-row">
                        <span>D / Right</span>
                        <span class="key-cap">Steer R</span>
                    </div>
```

```
                    <div class="ctrl-row">
                        <span>ESC / P</span>
                        <span class="key-cap">Pause</span>
                    </div>
                </div>
            </div>
        </div>
    </div>

    <!-- Application modules linked in execution order -->
    <script src="handControl.js" defer></script>
    <script src="game.js" defer></script>
</body>
</html>
```

4.2. `style.css`

```
/* Cyberpunk 16-Bit Pixel Racing Style Guide for Eye-Controlled Car Dodger */

:root {
    --primary: #00f2fe;
    --secondary: #4facfe;
    --accent: #ef4444;
    --gold: #ffd700;
    --bg-dark: #000000;
    --panel-bg: rgba(6, 8, 16, 0.82);
    --border-glow: rgba(0, 242, 254, 0.2);
    --font: 'Press Start 2P', monospace, cursive;
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: var(--font);
}

body {
    background-color: #000000;
    background: url('background.jpg') center center / cover no-repeat fixed;
    color: #ffffff;
    font-family: var(--font);
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    max-height: 100vh;
    margin: 0;
    padding: 0;
    overflow: hidden;
}

/* Gold Headings & Accents */
.gold-heading {
    color: #ffd700 !important;
    text-shadow: 0 0 8px rgba(255, 215, 0, 0.4);
}

/* Main Layout container */
#app-container {
    display: flex;
    flex-direction: row;
    gap: 24px;
    align-items: center;
    justify-content: center;
    position: relative;
    max-width: 1020px;
    width: 100%;
    margin: auto;
    padding: 12px;
    box-sizing: border-box;
}

/* Main Game screen layout */
#game-window-wrapper {
    position: relative;
    width: 470px;
    height: 630px;
```

```
    border-radius: 0px;
    overflow: hidden;
    box-shadow: 0 20px 50px rgba(0, 0, 0, 0.9), 0 0 0 1px rgba(255, 255, 255, 0.15);
    background-color: rgba(6, 8, 16, 0.82);
    flex-shrink: 0;
}

#gameCanvas {
    display: block;
    width: 470px;
    height: 630px;
    background-color: #1c1d22;
}

/* Screens Overlaid on Canvas */
.menu-overlay {
    position: absolute;
    top: 0;
    left: 0;
    width: 470px;
    height: 630px;
    background: rgba(6, 8, 16, 0.88);
    backdrop-filter: blur(4px);
    -webkit-backdrop-filter: blur(4px);
    border-radius: 0px;
    border: 1px solid rgba(255, 255, 255, 0.15);
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    padding: 24px;
    z-index: 10;
    opacity: 0;
    pointer-events: none;
    transition: opacity 0.3s ease, transform 0.3s ease;
    transform: scale(0.98);
    text-align: center;
}

.menu-overlay.active {
    opacity: 1;
    pointer-events: auto;
    transform: scale(1);
}

/* Typography & Visual Accents */
h1 {
    font-size: 1.5rem;
    font-weight: 800;
    margin-bottom: 12px;
    color: #ffd700;
    text-shadow: 0 0 12px rgba(255, 215, 0, 0.45);
    text-transform: uppercase;
    letter-spacing: 1px;
    line-height: 1.3;
}

p.subtitle {
    font-size: 0.65rem;
    color: #ffffff;
    margin-bottom: 20px;
    line-height: 1.5;
}

h2 {
```

```
    font-size: 1.25rem;
    font-weight: 700;
    margin-bottom: 12px;
    color: #ffd700;
    text-shadow: 0 0 10px rgba(255, 215, 0, 0.35);
    line-height: 1.3;
}

/* Cyber Button Styling */
.btn-container {
    display: flex;
    flex-direction: column;
    gap: 14px;
    width: 100%;
    max-width: 320px;
}

.cyber-btn {
    position: relative;
    padding: 14px 20px;
    font-size: 0.72rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    line-height: 1.4;
    color: #ffffff;
    background: #000000;
    border: 2px solid var(--primary);
    border-radius: 0px;
    cursor: pointer;
    overflow: hidden;
    transition: all 0.25s ease;
    box-shadow: 0 0 10px rgba(0, 242, 254, 0.15);
}

.cyber-btn::before {
    content: '';
    position: absolute;
    top: 0;
    left: -100%;
    width: 100%;
    height: 100%;
    background: linear-gradient(90deg, transparent, rgba(255, 255, 255, 0.2), transparent);
    transition: left 0.4s ease;
}

.cyber-btn:hover::before {
    left: 100%;
}

.cyber-btn.primary {
    background: linear-gradient(135deg, var(--primary), var(--secondary));
    color: #000000;
    border-color: transparent;
    box-shadow: 0 4px 15px rgba(0, 242, 254, 0.4);
}

.cyber-btn.primary:hover {
    box-shadow: 0 4px 25px rgba(0, 242, 254, 0.7);
    transform: translateY(-2px);
}

.cyber-btn.secondary {
    border-color: #475569;
    color: #ffffff;
```

```
}

.cyber-btn.secondary:hover {
    border-color: #94a3b8;
    color: #ffffff;
    background: rgba(255, 255, 255, 0.15);
}

/* HUD Overlay on playing canvas */
#game-hud {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    padding: 12px 16px;
    display: flex;
    justify-content: space-between;
    align-items: flex-start;
    pointer-events: none;
    z-index: 5;
}

.hud-panel {
    background: rgba(0, 0, 0, 0.85);
    border: 1px solid rgba(255, 255, 255, 0.15);
    padding: 6px 12px;
    border-radius: 0px;
    display: flex;
    flex-direction: column;
}

.hud-label {
    font-size: 0.55rem;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    color: #ffd700;
    margin-bottom: 4px;
}

.hud-val {
    font-size: 0.85rem;
    font-weight: 700;
    color: #ffffff;
}

/* Control Panel widget (Right sidebar) */
#control-sidebar {
    width: 470px;
    height: 630px;
    background: rgba(6, 8, 16, 0.82);
    backdrop-filter: blur(4px);
    -webkit-backdrop-filter: blur(4px);
    border: 1px solid rgba(255, 255, 255, 0.15);
    border-radius: 0px;
    padding: 18px;
    box-shadow: 0 20px 50px rgba(0, 0, 0, 0.9), 0 0 0 1px rgba(255, 255, 255, 0.15);
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    gap: 14px;
    box-sizing: border-box;
    flex-shrink: 0;
}

#control-sidebar.in-game {
```

```
    justify-content: flex-start;
}

#control-sidebar.in-game #instructions-section {
    display: none;
}

#control-sidebar.in-game #webcam-preview-container {
    display: block;
    flex: 1;
    height: 100%;
    margin-top: 4px;
}

.panel-header {
    border-bottom: 1px solid rgba(255, 255, 255, 0.15);
    padding-bottom: 10px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.panel-title {
    font-size: 0.82rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    color: #ffd700;
}

/* Badges styling */
.badge {
    padding: 4px 8px;
    border-radius: 0px;
    font-size: 0.55rem;
    font-weight: 700;
    text-transform: uppercase;
}

.badge.keyboard {
    background: rgba(71, 85, 105, 0.4);
    border: 1px solid #475569;
    color: #ffffff;
}

.badge.hand {
    background: rgba(16, 185, 129, 0.2);
    border: 1px solid #10b981;
    color: #34d399;
}

/* Webcam Preview area */
#webcam-preview-container {
    position: relative;
    width: 100%;
    height: 100%;
    border-radius: 0px;
    overflow: hidden;
    background: #000000;
    border: 1px solid rgba(255, 255, 255, 0.15);
    display: none;
}

#webcam-video {
    width: 100%;
```

```
    height: 100%;
    object-fit: cover;
    transform: scaleX(-1);
    display: none;
}

#webcam-overlay {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    object-fit: cover;
}

/* Instructions section wrapper */
#instructions-section {
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    gap: 12px;
    flex: 1;
}

/* How to Play Container */
.how-to-play-container {
    background: rgba(255, 255, 255, 0.03);
    border-left: 3px solid #ffd700;
    padding: 12px 14px;
    flex-grow: 1;
    display: flex;
    flex-direction: column;
    justify-content: center;
}

.how-to-play-list {
    list-style-type: square;
    padding-left: 14px;
    color: #ffffff;
    font-size: 0.58rem;
    line-height: 1.7;
}

.how-to-play-list li {
    margin-bottom: 6px;
}

.how-to-play-list li strong {
    color: #ffd700;
}

/* Controls list display */
.keyboard-controls-guide {
    background: rgba(0, 0, 0, 0.4);
    border: 1px solid rgba(255, 255, 255, 0.08);
    padding: 10px 14px;
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.guide-title {
    font-size: 0.65rem;
    font-weight: 700;
    margin-bottom: 2px;
```

```
}

.ctrl-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.6rem;
    color: #ffffff;
}

.key-cap {
    background: #111827;
    border: 1px solid #4b5563;
    border-radius: 0px;
    padding: 3px 8px;
    font-size: 0.55rem;
    font-weight: 700;
    color: #ffd700;
}
```

4.3. `game.js`

```
// Game Engine Logic for Hand-Controlled Car Dodger

const canvas = document.getElementById('gameCanvas');
const ctx = canvas.getContext('2d');

// Game State Constants
const STATES = {
    MENU: 'MENU',
    PLAYING: 'PLAYING',
    PAUSED: 'PAUSED',
    GAMEOVER: 'GAMEOVER'
};

let gameState = STATES.MENU;

// Speed Progression (50 -> 75 -> 90 -> 100 mph max)
const SPEED_STAGES = [50, 75, 90, 100];
let currentSpeedMph = 50;
let targetSpeedMph = 50;
let speedStage = 0;
let stageTimer = 0;

// High Score management
let highScore = parseInt(localStorage.getItem('carDodgerHighScore') || '0');

// Input Settings
let controlMode = 'KEYBOARD'; // 'KEYBOARD' or 'HAND'

// Road Design (Scaled for 470x630 canvas)
const ROAD = {
    x: 75,
    width: 320,
    lanes: [128, 235, 342], // Center X of each lane
    laneWidth: 107,
    speed: 5,
    maxSpeed: 10,
    offset: 0
};

// Player Object (Slightly smaller, centered)
const player = {
    lane: 1, // Start in Center lane
    x: ROAD.lanes[1] - 16,
    targetX: ROAD.lanes[1] - 16,
    y: 510,
    width: 32,
    height: 62,
    color: '#00f2fe',
    lerpSpeed: 0.18, // Smoothness of steering
    score: 0,
    reset() {
        this.lane = 1;
        this.targetX = ROAD.lanes[1] - this.width / 2;
        this.x = this.targetX;
        this.y = 510;
        this.score = 0;
    }
};

// Enemies List
let enemies = [];
```

```
let enemySpawnTimer = 0;
let enemySpawnInterval = 120;
const minSpawnInterval = 45;

// Particle Explosions
let particles = [];

// Keyboard listener setup
const keys = {};
window.addEventListener('keydown', (e) => {
    keys[e.code] = true;

    // State-independent hotkeys
    if (e.code === 'KeyP' || e.code === 'Escape') {
        if (gameState === STATES.PLAYING) {
            gameState = STATES.PAUSED;
            showUIOverlay('paused-screen');
        } else if (gameState === STATES.PAUSED) {
            gameState = STATES.PLAYING;
            hideUIOverlays();
        }
    }

    // Quick steering tests (Keyboard fallback)
    if (gameState === STATES.PLAYING && controlMode === 'KEYBOARD') {
        if (e.code === 'ArrowLeft' || e.code === 'KeyA') {
            steerLeft();
        }
        if (e.code === 'ArrowRight' || e.code === 'KeyD') {
            steerRight();
        }
    }
});
window.addEventListener('keyup', (e) => {
    keys[e.code] = false;
});

// Steering logic
function steerLeft() {
    if (player.lane > 0) {
        player.lane--;
        player.targetX = ROAD.lanes[player.lane] - player.width / 2;
    }
}

function steerRight() {
    if (player.lane < 2) {
        player.lane++;
        player.targetX = ROAD.lanes[player.lane] - player.width / 2;
    }
}

// Map detected hand zone to lane selection
function handleHandSteering() {
    if (window.handControl && window.handControl.isActive && window.handControl.handDetected) {
        const zone = window.handControl.currentZone;
        let targetLane = player.lane;

        if (zone === 'LEFT') {
            targetLane = 0;
        } else if (zone === 'CENTER') {
            targetLane = 1;
        } else if (zone === 'RIGHT') {
            targetLane = 2;
        }
```

```
        if (targetLane !== player.lane) {
            player.lane = targetLane;
            player.targetX = ROAD.lanes[player.lane] - player.width / 2;
        }
    }
}

// Enemy Spawning and management (Cars, SUVs, Pickups, Trucks - No motorcycles)
function spawnEnemy() {
    const lane = Math.floor(Math.random() * 3);

    const types = ['CAR', 'SUV', 'PICKUP', 'TRUCK'];
    const type = types[Math.floor(Math.random() * types.length)];

    let width, height;
    if (type === 'CAR') {
        width = 32;
        height = 62;
    } else if (type === 'SUV') {
        width = 36;
        height = 68;
    } else if (type === 'PICKUP') {
        width = 36;
        height = 78;
    } else { // TRUCK
        width = 42;
        height = 105;
    }
    const x = ROAD.lanes[lane] - width / 2;

    // Randomize colors
    const colors = ['#ef4444', '#a855f7', '#f59e0b', '#3b82f6', '#10b981', '#ec4899'];
    const color = colors[Math.floor(Math.random() * colors.length)];

    const speedOffset = (type === 'TRUCK') ? -1 : ((type === 'CAR') ? 0.5 : 0);

    enemies.push({
        x: x,
        y: -height - 20,
        width: width,
        height: height,
        color: color,
        speed: ROAD.speed + speedOffset + (Math.random() * 1.5 - 0.5),
        lane: lane,
        type: type
    });
}

// Particle System for collision explosions
function createExplosion(x, y, color) {
    const count = 30;
    for (let i = 0; i < count; i++) {
        particles.push({
            x: x,
            y: y,
            vx: (Math.random() - 0.5) * 8,
            vy: (Math.random() - 0.5) * 8,
            size: Math.random() * 4 + 2,
            alpha: 1,
            decay: Math.random() * 0.03 + 0.01,
            color: color
        });
    }
}
```

```
function updateParticles() {
    for (let i = particles.length - 1; i >= 0; i--) {
        const p = particles[i];
        p.x += p.vx;
        p.y += p.vy;
        p.alpha -= p.decay;
        if (p.alpha <= 0) {
            particles.splice(i, 1);
        }
    }
}

function drawParticles() {
    particles.forEach(p => {
        ctx.save();
        ctx.globalAlpha = p.alpha;
        ctx.fillStyle = p.color;
        ctx.beginPath();
        ctx.arc(p.x, p.y, p.size, 0, Math.PI * 2);
        ctx.fill();
        ctx.restore();
    });
}

// Collision helper using Axis-Aligned Bounding Box (AABB)
function checkCollision(r1, r2) {
    const padding = 3;
    return r1.x + padding < r2.x + r2.width - padding &&
           r1.x + r1.width - padding > r2.x + padding &&
           r1.y + padding < r2.y + r2.height - padding &&
           r1.y + r1.height - padding > r2.y + padding;
}

// Main Draw Functions
function drawRoad() {
    // Green grass backgrounds
    ctx.fillStyle = '#1e3a1f';
    ctx.fillRect(0, 0, ROAD.x, canvas.height);
    ctx.fillRect(ROAD.x + ROAD.width, 0, canvas.width - (ROAD.x + ROAD.width), canvas.height);

    // Asphalt
    ctx.fillStyle = '#22252a';
    ctx.fillRect(ROAD.x, 0, ROAD.width, canvas.height);

    // Shoulders (white lines)
    ctx.fillStyle = '#ffffff';
    ctx.fillRect(ROAD.x - 2, 0, 4, canvas.height);
    ctx.fillRect(ROAD.x + ROAD.width - 2, 0, 4, canvas.height);

    // Animated dashed lane lines
    ctx.save();
    ctx.strokeStyle = 'rgba(255, 255, 255, 0.4)';
    ctx.lineWidth = 4;
    ctx.setLineDash([30, 40]);
    ctx.lineDashOffset = -ROAD.offset;

    // Left Divider
    const div1 = ROAD.x + ROAD.laneWidth;
    ctx.beginPath();
    ctx.moveTo(div1, 0);
    ctx.lineTo(div1, canvas.height);
    ctx.stroke();

    // Right Divider
```

```
    const div2 = ROAD.x + ROAD.laneWidth * 2;
    ctx.beginPath();
    ctx.moveTo(div2, 0);
    ctx.lineTo(div2, canvas.height);
    ctx.stroke();
    ctx.restore();
}

function drawPlayerCar(x, y, w, h) {
    ctx.save();

    // Shadows
    ctx.shadowBlur = 10;
    ctx.shadowColor = 'rgba(0, 242, 254, 0.45)';

    // Main body gradient (Cyan sports car)
    const bodyGrad = ctx.createLinearGradient(x, y, x, y + h);
    bodyGrad.addColorStop(0, '#00f2fe');
    bodyGrad.addColorStop(1, '#0284c7');

    // Wheels (4 corners)
    ctx.fillStyle = '#0b0f19';
    ctx.beginPath();
    ctx.roundRect(x - 3, y + 6, 4, 13, 2);
    ctx.roundRect(x + w - 1, y + 6, 4, 13, 2);
    ctx.roundRect(x - 3, y + h - 19, 4, 13, 2);
    ctx.roundRect(x + w - 1, y + h - 19, 4, 13, 2);
    ctx.fill();

    // Main Chassis
    ctx.fillStyle = bodyGrad;
    ctx.beginPath();
    ctx.roundRect(x, y, w, h, 6);
    ctx.fill();

    // Windshield (Dark glossy cockpit)
    ctx.fillStyle = '#0b1329';
    ctx.beginPath();
    ctx.roundRect(x + 4, y + 16, w - 8, h - 38, [5, 5, 2, 2]);
    ctx.fill();

    // Glare on windshield
    ctx.strokeStyle = 'rgba(255, 255, 255, 0.3)';
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(x + 7, y + 19);
    ctx.lineTo(x + w - 9, y + 27);
    ctx.stroke();

    // Headlight glowing bulbs
    ctx.shadowBlur = 12;
    ctx.shadowColor = '#00f2fe';
    ctx.fillStyle = '#e0f2fe';
    ctx.beginPath();
    ctx.arc(x + 6, y + 2, 2.5, 0, Math.PI * 2);
    ctx.arc(x + w - 6, y + 2, 2.5, 0, Math.PI * 2);
    ctx.fill();

    // Headlight Beams (translucent polygons)
    const beamGrad = ctx.createLinearGradient(x, y, x, y - 50);
    beamGrad.addColorStop(0, 'rgba(0, 242, 254, 0.35)');
    beamGrad.addColorStop(1, 'rgba(0, 242, 254, 0)');
    ctx.fillStyle = beamGrad;
    ctx.shadowBlur = 0;
```

```
    // Left beam
    ctx.beginPath();
    ctx.moveTo(x + 3, y);
    ctx.lineTo(x - 12, y - 50);
    ctx.lineTo(x + 16, y - 50);
    ctx.closePath();
    ctx.fill();

    // Right beam
    ctx.beginPath();
    ctx.moveTo(x + w - 3, y);
    ctx.lineTo(x + w - 16, y - 50);
    ctx.lineTo(x + w + 12, y - 50);
    ctx.closePath();
    ctx.fill();

    // Spoiler / Rear Wing
    ctx.fillStyle = '#0f172a';
    ctx.fillRect(x - 2, y + h - 5, w + 4, 3);
    ctx.fillStyle = '#00f2fe';
    ctx.fillRect(x - 2, y + h - 5, 2.5, 5);
    ctx.fillRect(x + w - 0.5, y + h - 5, 2.5, 5);

    ctx.restore();
}

function drawEnemyCar(enemy) {
    const { x, y, width, height, color, type } = enemy;
    ctx.save();

    ctx.shadowBlur = 8;
    ctx.shadowColor = 'rgba(0, 0, 0, 0.4)';

    // Wheels for all 4-wheel vehicles
    ctx.fillStyle = '#111827';
    ctx.beginPath();
    ctx.roundRect(x - 3, y + 7, 4, 13, 2);
    ctx.roundRect(x + width - 1, y + 7, 4, 13, 2);
    ctx.roundRect(x - 3, y + height - 20, 4, 13, 2);
    ctx.roundRect(x + width - 1, y + height - 20, 4, 13, 2);
    ctx.fill();

    if (type === 'TRUCK') {
        // Truck Extra Middle Wheels
        ctx.beginPath();
        ctx.roundRect(x - 3, y + height - 42, 4, 13, 2);
        ctx.roundRect(x + width - 1, y + height - 42, 4, 13, 2);
        ctx.fill();

        // Semi Cabin
        ctx.fillStyle = '#1e293b';
        ctx.beginPath();
        ctx.roundRect(x, y + height - 32, width, 32, 3);
        ctx.fill();

        // Cargo Trailer
        const trailerGrad = ctx.createLinearGradient(x, y + height - 34, x, y);
        trailerGrad.addColorStop(0, color);
        trailerGrad.addColorStop(1, '#1e1b4b');
        ctx.fillStyle = trailerGrad;
        ctx.beginPath();
        ctx.roundRect(x, y, width, height - 36, 2);
        ctx.fill();

        // Red Tail lights
```

```
        ctx.fillStyle = '#ef4444';
        ctx.shadowBlur = 8;
        ctx.shadowColor = '#ef4444';
        ctx.beginPath();
        ctx.arc(x + 7, y + height - 2, 3, 0, Math.PI * 2);
        ctx.arc(x + width - 7, y + height - 2, 3, 0, Math.PI * 2);
        ctx.fill();
    } else if (type === 'PICKUP') {
        // Pickup Body
        const bodyGrad = ctx.createLinearGradient(x, y + height, x, y);
        bodyGrad.addColorStop(0, color);
        bodyGrad.addColorStop(1, '#0f172a');
        ctx.fillStyle = bodyGrad;
        ctx.beginPath();
        ctx.roundRect(x, y, width, height, 5);
        ctx.fill();

        // Open Bed
        ctx.fillStyle = '#090d16';
        ctx.beginPath();
        ctx.roundRect(x + 3, y + 4, width - 6, height - 38, 2);
        ctx.fill();

        // Cabin Windshield
        ctx.fillStyle = '#1e293b';
        ctx.beginPath();
        ctx.roundRect(x + 4, y + height - 32, width - 8, 24, [2, 2, 4, 4]);
        ctx.fill();

        // Tail lights
        ctx.fillStyle = '#ef4444';
        ctx.shadowBlur = 8;
        ctx.shadowColor = '#ef4444';
        ctx.beginPath();
        ctx.arc(x + 5, y + height - 2, 2.5, 0, Math.PI * 2);
        ctx.arc(x + width - 5, y + height - 2, 2.5, 0, Math.PI * 2);
        ctx.fill();
    } else if (type === 'SUV') {
        // SUV Body (boxy, taller roof)
        const bodyGrad = ctx.createLinearGradient(x, y + height, x, y);
        bodyGrad.addColorStop(0, color);
        bodyGrad.addColorStop(1, '#18181b');
        ctx.fillStyle = bodyGrad;
        ctx.beginPath();
        ctx.roundRect(x, y, width, height, 6);
        ctx.fill();

        // Roof rack
        ctx.fillStyle = '#0f172a';
        ctx.fillRect(x + 4, y + 10, 2, height - 24);
        ctx.fillRect(x + width - 6, y + 10, 2, height - 24);

        // Windshield
        ctx.fillStyle = '#0b1329';
        ctx.beginPath();
        ctx.roundRect(x + 4, y + height - 34, width - 8, height - 44, [2, 2, 4, 4]);
        ctx.fill();

        // Tail lights
        ctx.fillStyle = '#ef4444';
        ctx.shadowBlur = 8;
        ctx.shadowColor = '#ef4444';
        ctx.beginPath();
        ctx.arc(x + 5, y + height - 2, 2.5, 0, Math.PI * 2);
        ctx.arc(x + width - 5, y + height - 2, 2.5, 0, Math.PI * 2);
```

```
        ctx.fill();
    } else { // Standard CAR
        const bodyGrad = ctx.createLinearGradient(x, y + height, x, y);
        bodyGrad.addColorStop(0, color);
        bodyGrad.addColorStop(1, '#000000');
        ctx.fillStyle = bodyGrad;
        ctx.beginPath();
        ctx.roundRect(x, y, width, height, 6);
        ctx.fill();

        // Windshield
        ctx.fillStyle = '#0f172a';
        ctx.beginPath();
        ctx.roundRect(x + 4, y + height - 34, width - 8, height - 42, [2, 2, 4, 4]);
        ctx.fill();

        // Tail Lights
        ctx.fillStyle = '#ef4444';
        ctx.shadowBlur = 8;
        ctx.shadowColor = '#ef4444';
        ctx.beginPath();
        ctx.arc(x + 5, y + height - 2, 2.5, 0, Math.PI * 2);
        ctx.arc(x + width - 5, y + height - 2, 2.5, 0, Math.PI * 2);
        ctx.fill();
    }

    ctx.restore();
}

// Update loop logic
function updateGame() {
    if (gameState !== STATES.PLAYING) return;

    // 1. Move road to simulate speed
    ROAD.offset = (ROAD.offset + ROAD.speed) % 70;

    // 2. Gradual Speed Progression (50 -> 75 -> 90 -> 100 mph max)
    if (speedStage < 3) {
        stageTimer += 1 / 60;
        if (stageTimer >= 15) {
            speedStage++;
            stageTimer = 0;
            targetSpeedMph = SPEED_STAGES[speedStage];
        }
    }

    // Smooth transition between speed levels
    currentSpeedMph += (targetSpeedMph - currentSpeedMph) * 0.015;
    if (currentSpeedMph > 100) currentSpeedMph = 100;

    // Map current mph to road speed (50 mph = 5, 100 mph = 10)
    ROAD.speed = (currentSpeedMph / 50) * 5;

    // Dynamic spawn interval based on score
    enemySpawnInterval = Math.max(minSpawnInterval, 120 - Math.floor(player.score / 5));

    // 3. Interpolate player steering
    player.x += (player.targetX - player.x) * player.lerpSpeed;

    // 4. Hand gesture steering
    if (controlMode === 'HAND') {
        handleHandSteering();
    }

    // 5. Spawn enemies
```

```
    enemySpawnTimer++;
    if (enemySpawnTimer >= enemySpawnInterval) {
        spawnEnemy();
        enemySpawnTimer = 0;
    }

    // 6. Update enemies
    for (let i = enemies.length - 1; i >= 0; i--) {
        const enemy = enemies[i];
        enemy.y += enemy.speed;

        // Collision Check
        if (checkCollision(player, enemy)) {
            triggerGameOver(enemy);
            return;
        }

        // Delete passed enemies, increment score
        if (enemy.y > canvas.height) {
            enemies.splice(i, 1);
            player.score++;
            updateScoreHUD();
        }
    }

    // Update HUD speed continuously for smooth numbers
    updateScoreHUD();

    // Update particles
    updateParticles();
}

// Handle Game HUD rendering
function updateScoreHUD() {
    document.getElementById('hud-score').textContent = player.score;
    document.getElementById('hud-speed').textContent = Math.round(currentSpeedMph) + ' mph';
}

// End game logic
function triggerGameOver(collidedEnemy) {
    gameState = STATES.GAMEOVER;
    createExplosion(player.x + player.width / 2, player.y + player.height / 2, player.color);
    createExplosion(collidedEnemy.x + collidedEnemy.width / 2, collidedEnemy.y + collidedEnemy.height
/ 2, collidedEnemy.color);

    // Save High Score
    if (player.score > highScore) {
        highScore = player.score;
        localStorage.setItem('carDodgerHighScore', highScore.toString());
    }

    setTimeout(() => {
        document.getElementById('over-score').textContent = player.score;
        document.getElementById('over-highscore').textContent = highScore;
        showUIOverlay('gameover-screen');
    }, 1000);
}

// Master Render loop
function draw() {
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    if (gameState === STATES.PLAYING || gameState === STATES.PAUSED || gameState === STATES.GAMEOVER)
{
        drawRoad();
```

```
        // Draw enemies
        enemies.forEach(drawEnemyCar);

        // Draw player (skip on gameover if exploded)
        if (gameState !== STATES.GAMEOVER || particles.length > 0) {
            drawPlayerCar(player.x, player.y, player.width, player.height);
        }

        // Draw particle explosion
        drawParticles();
    } else {
        // Draw clean road under menu for visuals
        drawRoad();
        ctx.fillStyle = 'rgba(10, 15, 26, 0.6)';
        ctx.fillRect(0, 0, canvas.width, canvas.height);
    }
}

// Master Frame loop
function frame() {
    updateGame();
    draw();
    requestAnimationFrame(frame);
}

// UI Modals triggers
function showUIOverlay(overlayId) {
    document.querySelectorAll('.menu-overlay').forEach(el => {
        el.classList.remove('active');
    });
    document.getElementById(overlayId).classList.add('active');
}

function hideUIOverlays() {
    document.querySelectorAll('.menu-overlay').forEach(el => {
        el.classList.remove('active');
    });
}

// Button Events / Action Links
document.getElementById('btn-play-keyboard').addEventListener('click', async () => {
    controlMode = 'KEYBOARD';
    document.getElementById('control-mode-badge').textContent = 'Keyboard';
    document.getElementById('control-mode-badge').className = 'badge keyboard';

    // Optional webcam background initialization for camera feed
    const video = document.getElementById('webcam-video');
    const overlay = document.getElementById('webcam-overlay');
    if (window.handControl && video && overlay) {
        window.handControl.init(video, overlay).then(ready => {
            if (ready) window.handControl.start();
        }).catch(() => {});
    }

    startGame();
});

document.getElementById('btn-play-hand').addEventListener('click', async () => {
    controlMode = 'HAND';
    document.getElementById('control-mode-badge').textContent = 'Hand Mode';
    document.getElementById('control-mode-badge').className = 'badge hand';

    const video = document.getElementById('webcam-video');
    const overlay = document.getElementById('webcam-overlay');
```

```
    const ready = await window.handControl.init(video, overlay);
    if (ready) {
        window.handControl.start();
        startGame();
    } else {
        alert('Webcam hand-tracking could not start. Reverting to keyboard control.');
        controlMode = 'KEYBOARD';
        document.getElementById('control-mode-badge').textContent = 'Keyboard';
        document.getElementById('control-mode-badge').className = 'badge keyboard';
        startGame();
    }
});

document.getElementById('btn-restart').addEventListener('click', () => {
    startGame();
});

document.getElementById('btn-resume').addEventListener('click', () => {
    gameState = STATES.PLAYING;
    hideUIOverlays();
});

function quitToMenu() {
    gameState = STATES.MENU;
    if (window.handControl) {
        window.handControl.stop();
    }

    // Switch sidebar back to How to Play menu mode
    const sidebar = document.getElementById('control-sidebar');
    if (sidebar) sidebar.classList.remove('in-game');
    const sidebarTitle = document.getElementById('sidebar-title');
    if (sidebarTitle) sidebarTitle.textContent = 'How to Play';

    document.getElementById('webcam-preview-container').style.display = 'none';
    document.getElementById('control-mode-badge').textContent = 'Keyboard';
    document.getElementById('control-mode-badge').className = 'badge keyboard';

    enemies = [];
    particles = [];
    player.reset();

    showUIOverlay('start-screen');
}

document.querySelectorAll('.btn-quit-action').forEach(btn => {
    btn.addEventListener('click', quitToMenu);
});

function startGame() {
    player.reset();
    enemies = [];
    particles = [];
    enemySpawnTimer = 0;
    enemySpawnInterval = 120;

    currentSpeedMph = 50;
    targetSpeedMph = 50;
    speedStage = 0;
    stageTimer = 0;
    ROAD.speed = 5;

    // Switch right sidebar to full in-game camera mode (hide instructions)
    const sidebar = document.getElementById('control-sidebar');
```

```
    if (sidebar) sidebar.classList.add('in-game');
    const sidebarTitle = document.getElementById('sidebar-title');
    if (sidebarTitle) sidebarTitle.textContent = (controlMode === 'HAND') ? 'Hand Control' : 'Camera
Feed';

    const webcamContainer = document.getElementById('webcam-preview-container');
    if (webcamContainer) webcamContainer.style.display = 'block';

    updateScoreHUD();
    hideUIOverlays();
    gameState = STATES.PLAYING;
}

// Real-time hook inside MediaPipe frame processor (Safely checking elements)
window.handControl.onFrame = () => {
    const valEl = document.getElementById('hand-pos-val');
    const zoneEl = document.getElementById('hand-zone-badge');
    const indEl = document.getElementById('hand-status-indicator');
    const statusEl = document.getElementById('hand-status-text');

    if (valEl) valEl.textContent = (window.handControl.rawX * 100).toFixed(0) + '%';
    if (zoneEl) zoneEl.textContent = window.handControl.currentZone;
    if (indEl && statusEl) {
        if (window.handControl.handDetected) {
            indEl.className = 'status-light detected';
            statusEl.textContent = 'Hand Tracking Active';
        } else {
            indEl.className = 'status-light lost';
            statusEl.textContent = 'Searching for Hand...';
        }
    }
};

// Fire master loop
frame();
```

4.4. `handControl.js`

```
// Hand Tracking & Gesture Steering Module using MediaPipe Hands

const handControl = {
    isActive: false,
    handDetected: false,
    currentZone: 'CENTER', // 'LEFT', 'CENTER', 'RIGHT'
    rawX: 0.5, // Normalized X position (0 to 1, mirrored for player perspective)
    rawY: 0.5,

    // Elements
    videoElement: null,
    canvasElement: null,
    canvasCtx: null,
    hands: null,
    camera: null,
    onFrame: null,

    // Thresholds with hysteresis
    thresholds: {
        leftBound: 0.38,
        rightBound: 0.62,
        hysteresis: 0.03
    },

    // Initialize webcam and MediaPipe Hands
    async init(videoEl, overlayCanvasEl) {
        this.videoElement = videoEl;
        this.canvasElement = overlayCanvasEl;
        this.canvasCtx = overlayCanvasEl.getContext('2d');

        if (typeof Hands === 'undefined') {
            console.error('MediaPipe Hands library is not loaded. Ensure CDN scripts are loaded in
index.html.');
            return false;
        }

        try {
            this.hands = new Hands({
                locateFile: (file) => `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`
            });

            this.hands.setOptions({
                maxNumHands: 1,
                modelComplexity: 1,
                minDetectionConfidence: 0.6,
                minTrackingConfidence: 0.6
            });

            this.hands.onResults((results) => this.onResults(results));

            this.camera = new Camera(this.videoElement, {
                onFrame: async () => {
                    if (this.isActive) {
                        await this.hands.send({ image: this.videoElement });
                    }
                },
                width: 320,
                height: 240
            });

            return true;
```

```
        } catch (err) {
            console.error('Error starting MediaPipe Hands:', err);
            return false;
        }
    },

    start() {
        if (this.camera) {
            this.isActive = true;
            this.camera.start();
        }
    },

    stop() {
        this.isActive = false;
        if (this.camera) {
            this.camera.stop();
        }
    },

    // Process Hand Landmarks
    onResults(results) {
        const ctx = this.canvasCtx;
        const w = this.canvasElement.width;
        const h = this.canvasElement.height;

        ctx.clearRect(0, 0, w, h);

        // Draw camera image mirrored
        ctx.save();
        ctx.translate(w, 0);
        ctx.scale(-1, 1);
        ctx.drawImage(results.image, 0, 0, w, h);
        ctx.restore();

        // Draw visual zone guides (Left, Center, Right)
        this.drawZoneGuides(ctx, w, h);

        if (results.multiHandLandmarks && results.multiHandLandmarks.length > 0) {
            this.handDetected = true;
            const landmarks = results.multiHandLandmarks[0];

            // In mirrored space: x_mirrored = 1 - x_raw
            // Landmark 9 is the Middle MCP (palm center), landmark 0 is wrist
            const palmCenter = landmarks[9];
            this.rawX = 1 - palmCenter.x;
            this.rawY = palmCenter.y;

            // Determine zone
            this.currentZone = this.calculateZone(this.rawX, this.currentZone);

            // Draw glowing hand skeleton in mirrored orientation
            this.drawHandSkeleton(ctx, landmarks, w, h);
        } else {
            this.handDetected = false;
            // Retain last zone or default to CENTER when hand removed
            this.currentZone = 'CENTER';
        }

        if (this.onFrame) {
            this.onFrame();
        }
    },

    // Calculate zone with hysteresis to prevent twitching near borders
```

```
    calculateZone(x, prevZone) {
        const { leftBound, rightBound, hysteresis } = this.thresholds;

        if (prevZone === 'LEFT') {
            if (x > leftBound + hysteresis) {
                return x > rightBound ? 'RIGHT' : 'CENTER';
            }
            return 'LEFT';
        } else if (prevZone === 'RIGHT') {
            if (x < rightBound - hysteresis) {
                return x < leftBound ? 'LEFT' : 'CENTER';
            }
            return 'RIGHT';
        } else { // prevZone === 'CENTER'
            if (x < leftBound - hysteresis) return 'LEFT';
            if (x > rightBound + hysteresis) return 'RIGHT';
            return 'CENTER';
        }
    },

    // Draw Left/Center/Right zone overlays on the camera preview
    drawZoneGuides(ctx, w, h) {
        const leftX = w * this.thresholds.leftBound;
        const rightX = w * this.thresholds.rightBound;

        // Highlight active zone
        ctx.fillStyle = 'rgba(0, 242, 254, 0.15)';
        if (this.currentZone === 'LEFT' && this.handDetected) {
            ctx.fillRect(0, 0, leftX, h);
        } else if (this.currentZone === 'CENTER' && this.handDetected) {
            ctx.fillRect(leftX, 0, rightX - leftX, h);
        } else if (this.currentZone === 'RIGHT' && this.handDetected) {
            ctx.fillRect(rightX, 0, w - rightX, h);
        }

        // Zone dividing lines
        ctx.strokeStyle = 'rgba(255, 255, 255, 0.25)';
        ctx.lineWidth = 1.5;
        ctx.setLineDash([4, 4]);

        ctx.beginPath();
        ctx.moveTo(leftX, 0);
        ctx.lineTo(leftX, h);
        ctx.moveTo(rightX, 0);
        ctx.lineTo(rightX, h);
        ctx.stroke();
        ctx.setLineDash([]);

        // Zone Text Labels
        ctx.font = "bold 10px 'Segoe UI', sans-serif";
        ctx.textAlign = 'center';
        ctx.fillStyle = this.currentZone === 'LEFT' && this.handDetected ? '#00f2fe' : 'rgba(255, 255,
255, 0.5)';
        ctx.fillText('LEFT', leftX / 2, 20);

        ctx.fillStyle = this.currentZone === 'CENTER' && this.handDetected ? '#00f2fe' : 'rgba(255,
255, 255, 0.5)';
        ctx.fillText('CENTER', (leftX + rightX) / 2, 20);

        ctx.fillStyle = this.currentZone === 'RIGHT' && this.handDetected ? '#00f2fe' : 'rgba(255,
255, 255, 0.5)';
        ctx.fillText('RIGHT', (rightX + w) / 2, 20);
    },

    // Draw futuristic neon cyber hand skeleton
```

```
    drawHandSkeleton(ctx, landmarks, w, h) {
        // Mirrored coordinate helper
        const getPt = (idx) => ({
            x: (1 - landmarks[idx].x) * w,
            y: landmarks[idx].y * h
        });

        // Skeleton connections
        const connections = [
            // Palm
            [0, 1], [0, 5], [5, 9], [9, 13], [13, 17], [0, 17],
            // Thumb
            [1, 2], [2, 3], [3, 4],
            // Index
            [5, 6], [6, 7], [7, 8],
            // Middle
            [9, 10], [10, 11], [11, 12],
            // Ring
            [13, 14], [14, 15], [15, 16],
            // Pinky
            [17, 18], [18, 19], [19, 20]
        ];

        // Draw glowing lines
        ctx.save();
        ctx.shadowBlur = 8;
        ctx.shadowColor = '#00f2fe';
        ctx.strokeStyle = 'rgba(0, 242, 254, 0.8)';
        ctx.lineWidth = 2.5;

        connections.forEach(([i, j]) => {
            const p1 = getPt(i);
            const p2 = getPt(j);
            ctx.beginPath();
            ctx.moveTo(p1.x, p1.y);
            ctx.lineTo(p2.x, p2.y);
            ctx.stroke();
        });

        // Draw joint nodes
        ctx.fillStyle = '#ffffff';
        landmarks.forEach((_, idx) => {
            const p = getPt(idx);
            const isFingertip = [4, 8, 12, 16, 20].includes(idx);
            ctx.beginPath();
            ctx.arc(p.x, p.y, isFingertip ? 4.5 : 2.5, 0, Math.PI * 2);
            ctx.fillStyle = isFingertip ? '#00f2fe' : '#ffffff';
            ctx.fill();
        });

        // Palm centroid reticle
        const centerPt = getPt(9);
        ctx.strokeStyle = '#ef4444';
        ctx.lineWidth = 2;
        ctx.beginPath();
        ctx.arc(centerPt.x, centerPt.y, 8, 0, Math.PI * 2);
        ctx.stroke();

        ctx.restore();
    }
};

window.handControl = handControl;
```

5. How to Run Locally

1. Start a local web server (required for webcam & MediaPipe CDN security policies):
   `python3 -m http.server 8080`
2. Open your browser at: `http://localhost:8080`
3. Grant webcam permissions when prompted.
4. Select **Play Hand Mode** to steer with hand gestures or **Play Keyboard** for keys!